

IBIS

Bringing Optionality to Python Dataframes



One Python dataframe API. Nearly 20 backends. Your choice, any time.

Your Name

Data in the D · 2026

What is a dataframe?

A dataframe is a table: rows are records, columns are named and typed. It's the shape underneath spreadsheets, CSVs, SQL query results — and most of the Python data stack.



Column

A named, typed field — e.g. `body_mass_g`: float64



Row

One record — e.g. one penguin

species	island	body_mass_g
Adelie	Torgersen	3750
Adelie	Biscoe	3800
Gentoo	Biscoe	5000

Every dataframe library represents this same shape.

Why dataframes took over



Tabular data is everywhere

Exports, logs, spreadsheets, CSVs, query results — most real-world data already looks like a table.



Expressive, chainable operations

filter, group, join, sort read like your train of thought — no manual loops or index bookkeeping.



One shape, many tools

The same mental model works in pandas, R, SQL, and Spark. Learn the shape once.

Where dataframes shine — and where they don't



Great fit

- Exploratory data analysis & ad hoc queries
- Cleaning and reshaping tabular data (ETL)
- Feature engineering for ML models
- Reporting & analytics on structured data



Reach for something else

- Deeply nested documents — flatten first, or use a document store
- Free-form text, images, audio — NLP/CV tooling, tensors
- Highly connected graph traversal — a graph database
- Sub-millisecond single-row lookups — an OLTP database

THE PROBLEM

Every Python dataframe library picks one engine for you



pandas

Single machine only.
Outgrow your laptop's memory?
Rewrite.



Polars

Blazing fast locally.
Not built to hand off to a cluster.



PySpark

Scales to a cluster.
Overkill — and its own API — for
small jobs.



Warehouse SQL

Powerful engine.
No Python dataframe
ergonomics at all.

Whichever you pick, switching later means rewriting your code from scratch.

What if you didn't have to choose?

Ibis is one Python dataframe API that runs on **nearly 20 backend engines** — DuckDB, Polars, Trino, Snowflake, BigQuery, PySpark, Postgres, and more.

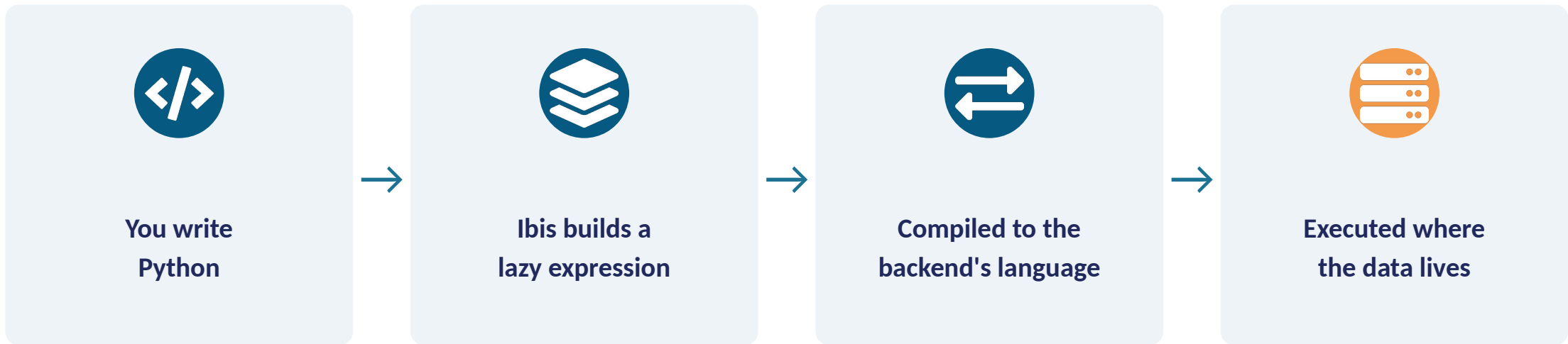
Learn the API once. Point it at whichever engine the job calls for — today, and again next year.

20+

backends, one API

DuckDB · Polars · Trino · Spark
Snowflake · BigQuery · Postgres ...

Your code compiles to wherever your data lives



Nothing touches your data until you call `.execute()` — that's what "lazy" means.

A handful of verbs cover almost everything

Ibis	What it does	Feels like
<code>select(...)</code>	Choose columns	<i>SQL SELECT</i>
<code>filter(...)</code>	Keep matching rows	<i>SQL WHERE</i>
<code>mutate(...)</code>	Add or transform a column	<i>computed column</i>
<code>group_by(...).aggregate(...)</code>	Summarize groups	<i>SQL GROUP BY</i>
<code>order_by(...)</code>	Sort rows	<i>SQL ORDER BY</i>
<code>join(...)</code>	Combine tables	<i>SQL JOIN</i>

Ibis vs. the dataframe libraries you already know

	pandas	Polars	PySpark	Ibis
One API, many engines	—	—	—	Yes — 20+
Runs locally, no server	Yes	Yes	Limited	Yes
Scales to a cluster	No	Not natively	Yes	Yes
Mixes SQL + dataframe code	No	Partial	Yes	Yes
Swap engines, keep your code	—	—	—	Yes

“Partial” / “limited” = supported in some form, but not the library's core design center.

Filtering rows, four APIs

pandas

```
df[
    df.mass > 4000
]
```

Polars

```
df.filter(
    pl.col("mass") >
    4000)
```

PySpark

```
df.filter(
    df.mass > 4000
)
```

Ibis

```
t.filter(
    t.mass > 4000
)
```

Same idea everywhere: keep the rows matching a condition. Ibis reads closest to pandas and PySpark.

Adding a computed column, four APIs

pandas

```
df.assign(  
    mass_kg=  
        df.mass/1000  
)
```

Polars

```
df.with_columns(  
    (pl.col("mass")  
     /1000).alias(  
        "mass_kg"))
```

PySpark

```
df.withColumn(  
    "mass_kg",  
    df.mass/1000  
)
```

Ibis

```
t.mutate(  
    mass_kg=  
        t.mass/1000  
)
```

Different verb per library (assign / with_columns / withColumn / mutate) — same shape of idea.

Group by and aggregate, four APIs

pandas

```
df.groupby("species")  
  ["mass"].mean()
```

Polars

```
df.group_by("species")  
  .agg(pl.col("mass")  
       .mean())
```

PySpark

```
df.groupBy("species")  
  .agg(F.mean("mass")  
  )
```

Ibis

```
t.group_by("species")  
  .aggregate(  
    t.mass.mean())
```

Ibis and Polars share the group_by spelling; the summarize-by-group idea is universal.

Joining two tables, four APIs

pandas

```
df.merge(  
    info, on="species"  
)
```

Polars

```
df.join(  
    info, on="species"  
)
```

PySpark

```
df.join(  
    info, on="species"  
)
```

Ibis

```
t.join(  
    info, "species"  
)
```

This is the closest four-way match — join syntax has mostly converged across modern dataframe APIs.

Mix SQL and dataframe code, freely, in either direction

Sometimes SQL is just the clearest way to say what you mean. Ibis lets you **drop into raw SQL** and pick the dataframe API back up on the result — or go the other way with `ibis.to_sql()`.

```
# Start in SQL...
by_island = con.sql("""SELECT species, island, count(*) AS n
                        FROM penguins GROUP BY 1, 2""")

# ...finish in the dataframe API
by_island.mutate(pct=by_island.n / by_island.n.sum())
               .order_by(ibis.desc("n"))
```

No need to pick a camp — use whichever reads more clearly for that step.

DuckDB on my laptop → Trino on a real cluster



Write the analysis

against a local DuckDB connection



Run it — it works

fast, local, zero infra



Change one connection

point Ibis at Trino instead



Same code, new engine

now running as a distributed query

Follow along: `ibis_optionality_demo.ipynb`

Local dev: connect, build, run — on DuckDB

```
con = ibis.duckdb.connect()
penguins = con.read_parquet("penguins.parquet")

def top_species_by_mass(t):
    return (t.filter(t.body_mass_g.notnull())
            .group_by("species")
            .aggregate(avg_mass_g=t.body_mass_g.mean())
            .order_by(ibis.desc("avg_mass_g")))

top_species_by_mass(penguins)
```

Same function, new engine — nothing else changes

```
# — the one thing that changes —  
con = ibis.trino.connect(host=..., port=443, user=...,  
                        database="hive", schema="default")  
  
penguins = con.table("penguins")  
  
# identical to the DuckDB slide — copy-pasted, unchanged  
top_species_by_mass(penguins) ← now a distributed Trino query
```

top_species_by_mass never finds out which engine it's running on. That's the optionality.

WHY IT MATTERS

Optionality is leverage — for your project, and your career



Prototype fast, ship without rewrites

Build and test locally on DuckDB, then point at production infrastructure with a config change, not a rewrite.



Never married to one platform

Your team's stack changes. Ibis code mostly doesn't have to.



One skill, every employer's stack

Whatever warehouse or engine your next job runs, there's a good chance Ibis already speaks to it.

Run this exact demo yourself, tonight or next week

-  **1. Clone the repo & install**

```
pip install -r requirements.txt
```
-  **2. Cache the demo dataset**

```
python prep_offline_data.py
```

 (works fully offline after this)
-  **3. Get a Trino endpoint**
Local via Docker Compose, or a free Starburst Galaxy cluster
-  **4. Open the notebook**

```
jupyter lab ibis_optionality_demo.ipynb
```

Full setup instructions, including both Trino options, are in SETUP.md.

Thank you

Questions welcome — find me after the session too.



Docs & tutorials

`ibis-project.org`



Source

`github.com/ibis-project/ibis`



This talk's demo + slides

`github.com/your-handle/ibis-optionality-talk`